

Building API Views (CRUD)

Lecture 10

FBV • CBV • Mixins • Generic Views • Postman

KBTU — Web Development

1. Levels 0–1 — Raw Django (Recap)

These are what we built in previous weeks. Key pattern:

```
# Level 0: Manual serialization
@csrf_exempt
def products_list(request):
    if request.method == 'GET':
        products = Product.objects.all()
        products_json = [p.to_json() for p in products]    # manual
        return JsonResponse(products_json, safe=False)
    elif request.method == 'POST':
        data = json.loads(request.body)                  # manual
        product = Product.objects.create(**data)
        return JsonResponse(product.to_json(), status=201)

# Level 1: Same but with DRF Serializer for validation
elif request.method == 'POST':
    data = json.loads(request.body)                      # still manual
    serializer = ProductSerializer(data=data)
    if serializer.is_valid():
        serializer.save()
    return JsonResponse(serializer.data, status=201)    # still JsonResponse
```

❌ Problems: `@csrf_exempt` (no security), `json.loads` (manual parsing), `JsonResponse` (no browsable API), if/elif chains.

2. Level 2 — DRF Function-Based Views

2.1 fbv.py — products_list

```
from rest_framework import status
from rest_framework.decorators import api_view
from rest_framework.response import Response
from api.models import Product
from api.serializers import ProductSerializer

@api_view(['GET', 'POST'])
def products_list(request):
    if request.method == 'GET':
        products = Product.objects.all()
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = ProductSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data,
                            status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
                        status=status.HTTP_400_BAD_REQUEST)
```

2.2 fbv.py — product_detail

```
@api_view(['GET', 'PUT', 'DELETE'])
def product_detail(request, product_id=None):
    try:
        product = Product.objects.get(pk=product_id)
    except Product.DoesNotExist as e:
        return Response({'error': str(e)},
```

```

        status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = ProductSerializer(product)
        return Response(serializer.data)
    elif request.method == 'PUT':
        serializer = ProductSerializer(instance=product,
                                      data=request.data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors,
                        status=status.HTTP_400_BAD_REQUEST)
    elif request.method == 'DELETE':
        product.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```

2.3 What Changed (Level 1 → 2)

Before	After	Why Better
@csrf_exempt	@api_view(['GET','POST'])	Proper security; auto-rejects unlisted methods (405)
json.loads(request.body)	request.data	Auto-parses JSON, form data, multipart
JsonResponse(data, safe=False)	Response(data)	Content negotiation + browsable API
status=201	status.HTTP_201_CREATED	Readable, self-documenting

3. Level 3 — Class-Based Views (APIView)

Replace if/elif with **separate methods** per HTTP verb.

3.1 cbv.py

```
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import APIView

class ProductListAPIView(APIView):
    def get(self, request):
        products = Product.objects.all()
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = ProductSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data,
                            status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
                        status=status.HTTP_400_BAD_REQUEST)

class ProductDetailAPIView(APIView):
    def get_object(self, product_id):
        try:
            return Product.objects.get(pk=product_id)
        except Product.DoesNotExist as e:
            return Response({'error': str(e)},
                            status=status.HTTP_404_NOT_FOUND)

    def get(self, request, product_id):
        product = self.get_object(product_id)
        serializer = ProductSerializer(product)
        return Response(serializer.data)

    def put(self, request, product_id):
        product = self.get_object(product_id)
        serializer = ProductSerializer(instance=product,
                                      data=request.data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors,
                        status=status.HTTP_400_BAD_REQUEST)

    def delete(self, request, product_id):
        product = self.get_object(product_id)
        product.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

3.2 URL Routing — .as_view()

```
# CBVs require .as_view() in urls.py
path('products/', ProductListAPIView.as_view()),
path('products/<int:product_id>', ProductDetailAPIView.as_view()),
```

3.3 FBV vs CBV

Aspect	FBV (@api_view)	CBV (APIView)
Method routing	if/elif on request.method	Separate get(), post(), put(), delete()
URL registration	path(..., view_func)	path(..., Class.as_view())
Shared logic	Separate helper function	self.get_object() on the class
Extensibility	Hard to inherit	Easy to subclass and override

4. Level 4 — Mixins + GenericAPIView

Mixins provide reusable CRUD operations. `GenericAPIView` adds `queryset` and `serializer_class`.

Mixin	Provides	Does
ListModelMixin	<code>self.list()</code>	GET list
CreateModelMixin	<code>self.create()</code>	POST create
RetrieveModelMixin	<code>self.retrieve()</code>	GET single
UpdateModelMixin	<code>self.update()</code>	PUT / PATCH
DestroyModelMixin	<code>self.destroy()</code>	DELETE

4.1 generic_v1.py

```
from rest_framework import generics, mixins

class ProductListAPIView(mixins.ListModelMixin,
                        mixins.CreateModelMixin,
                        generics.GenericAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)

class ProductDetailAPIView(mixins.RetrieveModelMixin,
                          mixins.UpdateModelMixin,
                          mixins.DestroyModelMixin,
                          generics.GenericAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    lookup_url_kwarg = 'product_id'

    def get(self, request, product_id):
        return self.retrieve(request, product_id)

    def put(self, request, product_id):
        return self.update(request, product_id)

    def delete(self, request, product_id):
        return self.destroy(request, product_id)
```

lookup_url_kwarg: Generic views default to 'pk'. Our URL uses 'product_id', so we set `lookup_url_kwarg = 'product_id'` to tell DRF.

Still writing stubs: `get()` just calls `self.list()`, `post()` calls `self.create()`. Can we eliminate these? → Level 5.

5. Level 5 — Concrete Generic Views

Generic View	Combines	HTTP Methods
ListCreateAPIView	List + Create + GenericAPIView	GET (list) + POST
RetrieveUpdateDestroyAPIView	Retrieve + Update + Destroy + GenericAPIView	GET + PUT + PATCH + DELETE
ListAPIView	List + GenericAPIView	GET (list only)
RetrieveAPIView	Retrieve + GenericAPIView	GET (detail only)

5.1 generic_v2.py

```
from rest_framework import generics, status
from rest_framework.response import Response
from rest_framework.views import APIView

class ProductListAPIView(generics.ListCreateAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

class ProductDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    lookup_url_kwarg = 'product_id'

class CategoryListAPIView(generics.ListCreateAPIView):
    queryset = Category.objects.all()
    serializer_class = CategorySerializer

class CategoryDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Category.objects.all()
    serializer_class = CategorySerializer
    lookup_url_kwarg = 'category_id'
```

5.2 Custom Endpoint (APIView fallback)

"Products for a category" doesn't fit standard CRUD → use APIView:

```
class CategoryProductsAPIView(APIView):
    def get(self, request, category_id):
        try:
            category = Category.objects.get(pk=category_id)
        except Category.DoesNotExist as e:
            return Response({'error': str(e)},
                            status=status.HTTP_404_NOT_FOUND)
        products = category.products.all()
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)
```

5.3 Final URLs

```
urlpatterns = [
    path('categories/', CategoryListAPIView.as_view()),
    path('categories/<int:category_id>/',
          CategoryDetailAPIView.as_view()),
    path('categories/<int:category_id>/products/',
          CategoryProductsAPIView.as_view()),
    path('products/', ProductListAPIView.as_view()),
    path('products/<int:product_id>/',
```

```
ProductDetailAPIView.as_view()),
```

```
]
```


6. The views/ Package Pattern

```
# api/views/__init__.py
# Swap imports to switch between levels:

# from .views_w11 import products_list, product_detail      # Level 1
from .fbv import products_list, product_detail              # Level 2
# from .cbv import ProductListAPIView, ProductDetailAPIView # Level 3
# from .generic_v1 import ProductListAPIView, ...           # Level 4
from .generic_v2 import ProductListAPIView, ProductDetailAPIView # Level 5
```

7. Testing with Postman & Browsable API

Operation	Method	URL	Expected
List	GET	/api/products/	200 + array
Create	POST	/api/products/	201 + object
Detail	GET	/api/products/1/	200 + object
Update	PUT	/api/products/1/	200 + updated
Delete	DELETE	/api/products/1/	204
Category products	GET	/api/categories/1/products/	200 + array

Browsable API: Open any DRF endpoint in a browser → interactive HTML page with forms. Free with Response.

Status Code Constants

Constant	Code	Use
HTTP_200_OK	200	GET, PUT success
HTTP_201_CREATED	201	POST success
HTTP_204_NO_CONTENT	204	DELETE success
HTTP_400_BAD_REQUEST	400	Validation error
HTTP_404_NOT_FOUND	404	Object not found

8. Side-by-Side: All Levels

Level	Lines	Key Difference
0: Raw Django	~50	to_json(), json.loads, @csrf_exempt, field-by-field
1: + Serializer	~35	Serializer for validation; still JsonResponse + json.loads
2: DRF FBV	~30	@api_view + request.data + Response
3: DRF CBV	~35	Class with separate methods; get_object() helper
4: Mixins	~18	queryset + serializer_class + self.list()/self.create()
5: Generics	~6	ListCreateAPIView / RetrieveUpdateDestroyAPIView

Summary

Topic	Key Takeaway
FBV (@api_view)	@api_view + request.data + Response — DRF with function style
CBV (APIView)	Class with get/post/put/delete; get_object() pattern; .as_view()
Mixins	ListModelMixin etc. + GenericAPIView; delegate to self.list()
Generic Views	ListCreateAPIView = GET+POST; RetrieveUpdateDestroyAPIView = full CRUD
Custom endpoints	Use APIView when it doesn't fit standard CRUD
views/ package	__init__.py swaps imports between levels
lookup_url_kwarg	Maps URL param name to object lookup
Browsable API	Free interactive UI in browser with any DRF view

References

- DRF Views — <https://www.django-rest-framework.org/api-guide/views/>
- DRF Generic Views — <https://www.django-rest-framework.org/api-guide/generic-views/>
- DRF Mixins — <https://www.django-rest-framework.org/api-guide/generic-views/#mixins>
- Postman — <https://www.postman.com/downloads/>

Live Coding Notes

From Raw Django to 3-Line Generics (~60 min)

Format: Refactor same Product CRUD through 4 levels. views/ package, swap imports in `__init__.py`.

Part	What	Time
Setup	Convert views.py → views/ package	3 min
1	Recap: show raw Django views, point out problems	5 min
2	FBV: create fbv.py + demo browsable API & Postman	15 min
3	CBV: create cbv.py, explain <code>get_object()</code> , <code>.as_view()</code>	12 min
4	Mixins: create <code>generic_v1.py</code> , explain mixin delegation	10 min
5	Generics: create <code>generic_v2.py</code> + Categories + custom endpoint	10 min
6	Wrap-up: show <code>__init__.py</code> , side-by-side comparison	5 min

Setup: views/ Package (3 min)

```
$ mkdir api/views  
$ mv api/views.py api/views/views_w11.py  
$ touch api/views/__init__.py
```

```
# api/views/__init__.py  
from .views_w11 import products_list, product_detail
```

Verify: Restart server, existing endpoints still work.

Part 1: Recap (5 min)

Open `views_w11.py`. Point out on screen:

- `@csrf_exempt` — disables security
- `json.loads(request.body)` — manual JSON parsing
- `JsonResponse` — no browsable API
- `if/elif` chains for method routing

Say: "Same functionality, progressively less code. Let's upgrade."

Part 2: FBV +Browsable API Demo (15 min)

Create `api/views/fbv.py`. Show key changes vs `views_w11.py`:

```
# KEY CHANGES (fbv.py vs views_w11.py):  
# 1. @csrf_exempt      → @api_view(['GET', 'POST'])  
# 2. json.loads(...)   → request.data  
# 3. JsonResponse(...) → Response(...)  
# 4. status=201        → status=status.HTTP_201_CREATED
```

Write both `products_list` and `product_detail` — same logic, DRF imports.

Swap imports

```
# api/views/__init__.py  
from .fbv import products_list, product_detail
```

Demo: Browsable API

Open `http://127.0.0.1:8000/api/products/` in browser — HTML page! POST via form. Click detail link — PUT/DELETE.

Demo: Postman CRUD

GET → 200, POST `{"name":"iPhone","price":999.99,"category_id":1}` → 201, PUT → 200, DELETE → 204, GET `/999/` → 404.

Part 3: CBV (12 min)

Create `api/views/cbv.py`. Key structural change:

```
class ProductListAPIView(APIView):
    def get(self, request):      # handles GET
    ...
    def post(self, request):     # handles POST
    ...

class ProductDetailAPIView(APIView):
    def get_object(self, product_id): # shared helper
        try: return Product.objects.get(pk=product_id)
        except Product.DoesNotExist as e:
            return Response({'error': str(e)}, status=...)

    def get(self, request, product_id): ...
    def put(self, request, product_id): ...
    def delete(self, request, product_id): ...
```

Fill in method bodies — same serializer logic as FBV, organized into methods.

Update `urls.py`

```
# .as_view() required for classes!
path('products/', ProductListAPIView.as_view()),
path('products/<int:product_id>', ProductDetailAPIView.as_view()),
```

Test: Same Postman requests. "Zero behavior change, cleaner structure."

Part 4: Mixins (10 min)

Say: "Every list view has the same get/post. Mixins do this for us."

Create `api/views/generic_v1.py`:

```
class ProductListAPIView(mixins.ListModelMixin,
                        mixins.CreateModelMixin,
                        generics.GenericAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)
    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)

class ProductDetailAPIView(mixins.RetrieveModelMixin,
                          mixins.UpdateModelMixin,
                          mixins.DestroyModelMixin,
                          generics.GenericAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    lookup_url_kwarg = 'product_id'

    def get(self, request, pid): return self.retrieve(request, pid)
    def put(self, request, pid): return self.update(request, pid)
    def delete(self, request, pid): return self.destroy(request, pid)
```

Explain `lookup_url_kwarg`: URL has 'product_id', generics default to 'pk'. This maps them.

Say: "Still writing stubs. Can we remove those too?"

Part 5: Concrete Generics + Categories (10 min)

Create `api/views/generic_v2.py`:

```
class ProductListAPIView(generics.ListCreateAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

class ProductDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    lookup_url_kwarg = 'product_id'

# Same for Category + custom endpoint:
class CategoryProductsAPIView(APIView):
    def get(self, request, category_id):
        category = Category.objects.get(pk=category_id)
        products = category.products.all()
        return Response(ProductSerializer(products, many=True).data)
```

Update `urls.py` with all 5 endpoints. Test categories.

Key: "CategoryProductsAPIView uses APIView—non-standard CRUD needs custom logic."

Part 6: Wrap-up (5 min)

Show `__init__.py` with all imports:

```
# api/views/__init__.py
# from .views_w11 import ...      # Level 1
from .fbv import ...              # Level 2
# from .cbv import ...           # Level 3
# from .generic_v1 import ...    # Level 4
from .generic_v2 import ...       # Level 5 (active)
```

Level	Lines	Key Takeaway
0–1: Raw Django	35–50	Manual everything
2: FBV	~30	@api_view + request.data + Response
3: CBV	~35	Separate methods; get_object()
4: Mixins	~18	Mixin delegation
5: Generics	~6	3 lines per resource

Close: "50 lines → 6 lines. Same API. Know all levels—debug through the layers."